

Proyecto: Sistema de Gestión de Inventarios para Pequeñas Empresas orientado a Restaurante.

Ingeniería de Requerimientos | Arquitectura de Software | Modelado Estructural y de Interacción | Patrones Creacionales | Patrones Estructurales | Patrones de Comportamiento | Modelo Computacional | Diseño de Interfaces

Equipo: Grupo 3: "Los Sin Nombre"
Integrantes: Miguel Aliaga, Esteban Guzman, Fabian Paredes
Asignatura: Diseño de Software - Profesor: Ernesto Pereiras García
Universidad de Talca - Facultad de Ingeniería



Contexto, Problema y Solución Propuesta

Contexto: Alta complejidad operativa y naturaleza rápida del rubro gastronómico.

El Problema: Uso de métodos manuales o sistemas genéricos no adaptados.

La Solución: Aplicación web centralizada que automatiza y asegura la trazabilidad del ciclo de vida del producto (desde la compra al proveedor hasta el registro de la venta).

Objetivos y Requerimientos del Sistema

Objetivo General: Desarrollar una aplicación web centralizada cliente-servidor que optimice el inventario, ventas y proveedores, controlando el stock en tiempo real mediante roles seguros.

Requerimientos Funcionales (RF):

- RF-01 (Actualizar): Modificar la cantidad de stock en ≤ 2 segundos tras una venta.
- RF-02 (Consultar): Desplegar la lista de productos organizada por categorías.

Requerimientos No Funcionales (RNF):

- NF-UX-01 (Usabilidad): Flujo de venta rápido completado en ≤ 5 clics.
- NF-MD-02 (Modularidad): Incorporar o modificar un rol afectando a ≤ 2 clases del sistema.

Metodología, Gestión del Proyecto y Distribución de Roles

Diagrama de clases UML:
UML

Metodología: Desarrollo incremental basado en hitos semanales guiados por laboratorios.

Distribución de Roles:

- Modelado Arquitectónico y UML: Diseño conceptual de la estructura de clases y componentes lógicos.
- Diseño de Interfaces (UI/UX): Prototipado interactivo enfocado en la baja carga cognitiva del usuario.
- Documentación y Pruebas: Análisis de requerimientos y simulación del modelo computacional.

Gestión de Tareas: Monitoreo constante de avances, mitigación de retrasos (como la curva de aprendizaje al implementar patrones complejos) y definición de próximos pasos.

Arquitectura del Sistema (Lógica y Física)

Arquitectura Lógica: Organización en capas bien definidas (Presentación/Frontend, Negocio/Servidor Principal y Capa de Datos/Persistencia).

Patrón Arquitectónico: Cliente-Servidor con Backend Monolítico.

Justificación Técnica:

- Evita inconsistencias de datos concurrentes al centralizar las transacciones.
- Garantiza que el descuento de stock se propague inmediatamente a todos los módulos.
- Idóneo para el tamaño del negocio, evitando la sobreingeniería de redes complejas.

Base de Datos: Modelo relacional en MySQL para asegurar la integridad de tablas como Usuarios, Productos, Ventas y Compras.

Patrones de Diseño Aplicados

Creacional: Singleton

Aplicado en: Clase Control (stock). Centraliza el estado único del inventario, impidiendo que existan copias desalineadas en memoria.

Estructural: Proxy

Aplicado en: Clase Venta (proxy) e interfaz ServicioVenta. Intercepta las solicitudes de facturación y valida mediante la clase Permiso si el usuario tiene el rol autorizado.

De Comportamiento: Observer

Aplicado en: Interfaces StockSubject y StockObserver. La clase Control notifica automáticamente a la clase Alerta al cruzar el stock mínimo, eliminando consultas repetitivas (polling).

Modelo Computacional e Interacciones API REST

Diseño Orientado a Recursos: Comunicación sincrónica HTTP utilizando payloads en formato JSON.

Endpoints Principales:

- GET /api/productos: Recupera el catálogo completo filtrado por categorías.
- POST /api/ventas: Envía los datos de la venta. Pasa por la validación del Proxy, ejecuta el INSERT en la base de datos y gatilla el UPDATE en el stock.
- GET /api/stock/alertas: Retorna la lista de productos críticos procesados por el Observer.

Repositorio Técnico y Plan de Pruebas

Github:
[Repositorio Github](#)

Estructura del Repositorio: Carpeta raíz organizada con entregables de laboratorios, archivos fuente del modelado UML (Lucidchart) y documentación técnica clara en un archivo README básico.

Plan de Pruebas y Validación:

- Pruebas de Usabilidad: Evaluación de la carga cognitiva mediante mapas de navegación anotados en Figma para validar el límite de ≤ 5 clics.
- Pruebas de Lógica Funcional: Casos de prueba diseñados para comprobar que el Proxy bloquee solicitudes no autorizadas y que el Singleton mantenga una sola instancia en accesos concurrentes.

Demostración del Prototipo Navegable (Figma)

Herramienta: Figma.

Enlace Público del Proyecto: [Figma Grupo 3](#)

Flujos Críticos a Demostrar:

1. Inicio de sesión y distribución visual según el rol del usuario.
2. Visualización inmediata de Alertas Críticas automatizadas en el Dashboard.
3. Registro expedito de transacciones reduciendo la carga cognitiva.

Conclusiones y Próximos Pasos

Aprendizajes Clave: El modelado arquitectónico previo mitiga errores costosos en la etapa de codificación. Los patrones reducen drásticamente el acoplamiento.

Riesgos Técnicos Detectados: El servidor centralizado representa un punto único de falla (si cae, se detiene la operación).

Próximos Pasos y Mitigación Futura:

- Diseñar un esquema de base de datos replicada.
- Desacoplar el módulo de reportes analíticos pesados hacia un microservicio independiente para proteger el rendimiento de las ventas en horarios de alta concurrencia.